

Advanced RAG Research Assistant

Full Project Report & Implementation Blueprint

PDF ingestion • Chunking • Embeddings • Hybrid Retrieval • Reranking • Citations • Multi-document
Comparison • Evaluation • Tracing • API • Docker

Completion checkpoint: Build one complete RAG application: ingestion → chunking → embeddings → retrieval → generation → evaluation/tracing → API → Docker.

1. Executive Summary

The Advanced RAG Research Assistant is an end-to-end retrieval-augmented generation system for research papers and other PDF documents. Users can upload documents, ask questions, compare multiple papers, and receive grounded answers with citations.

The project goes beyond a basic “chat with PDF” application by combining document processing, information retrieval, vector search, hybrid retrieval, reranking, LLM generation, evaluation, observability, API engineering, and deployment.

- PDF ingestion and text extraction
- Metadata-preserving chunking
- Embedding generation
- Dense/vector retrieval
- BM25 or keyword retrieval
- Hybrid retrieval
- Reranking
- Citation generation and validation
- Multi-document question answering
- Multi-document comparison
- Retrieval and generation evaluation
- Tracing and observability
- Evaluation dashboard
- REST API
- Dockerized deployment

2. Project Goal

Primary objective: Build an application that can answer questions from a collection of research papers while providing evidence for its answers.

```
PDFs
↓
Ingestion
↓
Chunking
↓
Embeddings
↓
Vector / keyword indexes
↓
Hybrid retrieval
↓
Reranking
↓
LLM
↓
Answer + citations
```

Example question: “Compare the training objectives of BERT, GPT-2 and LLaMA.” The system should retrieve evidence from the relevant papers, rank it, generate a structured comparison, and attach citations to the supporting evidence.

3. Why This Is a Strong Project

A basic RAG chatbot is common. This project is more valuable because it investigates the complete RAG lifecycle and makes retrieval quality measurable.

- NLP and embeddings
- Information retrieval
- Vector databases
- LLM application engineering
- Hybrid retrieval and reranking
- Evaluation methodology
- Backend/API engineering
- Observability and deployment

The goal is not merely to make a chatbot. The goal is to understand why every component exists and demonstrate measurable improvements when the architecture changes.

4. Final System Architecture



Evaluation
+
Tracing
+
Docker

5. Project Scope

A. Document ingestion

- PDF upload
- PDF parsing
- Text and page extraction
- Document metadata
- Document ID generation
- Duplicate detection

B. Chunking

- Fixed-size chunking
- Overlap
- Recursive/semantic strategies later
- Metadata-aware chunks

C. Embeddings

- Convert chunks into dense vectors
- Store embeddings with chunk metadata

D. Retrieval

- Dense vector retrieval
- Sparse/BM25 retrieval
- Hybrid retrieval

E. Reranking

- Retrieve a candidate set
- Rerank candidates
- Select high-quality evidence

F. Generation

- Grounded LLM answer generation
- Explicit insufficient-evidence behavior

G. Citations

- Document/page/chunk provenance
- Citation validation

H. Multi-document reasoning

- Cross-document questions

- Document comparison

I. Evaluation

- Evaluation dataset
- Retrieval metrics
- Generation metrics
- Citation accuracy

J. Observability

- Tracing
- Latency
- Token/cost tracking

K. API and deployment

- FastAPI
- Tests
- Docker
- Deployment

6. Prerequisites and Learning

Python

- Functions and classes
- File handling
- Exceptions
- Type hints
- Virtual environments
- JSON
- HTTP
- Async basics
- Git and package management

NLP and Embeddings

- Tokenization
- Embedding models
- Cosine similarity
- Dot product
- Vector representations
- Context windows

Information Retrieval

- TF-IDF concepts
- BM25

- Dense retrieval
- Sparse retrieval
- Top-K
- Recall and ranking

RAG

Understand the difference between direct LLM generation and retrieval-augmented generation. RAG retrieves external evidence at query time and provides that evidence to the generator.

7. Development Strategy

Do not build the complete system at once. Build progressively and freeze each working version before adding complexity.

Version	Objective
Version 0 — Environment	Create a clean Python project, dependency setup, Git repository, configuration, and environment-variable handling.
Version 1 — PDF Ingestion	Extract text, pages, and metadata from one PDF.
Version 2 — Chunking	Split extracted text into useful chunks and preserve metadata.
Version 3 — Embeddings	Generate vectors for chunks.
Version 4 — Vector Retrieval	Retrieve the top-K relevant chunks using dense similarity search.
Version 5 — Basic RAG	Send retrieved evidence to an LLM and generate a grounded answer.
Version 6 — Citations	Return document/page/chunk citations.
Version 7 — Multi-document RAG	Search across a collection of PDFs.
Version 8 — Metadata Filtering	Filter by document, year, topic, author, section, etc.
Version 9 — Hybrid Retrieval	Combine dense retrieval with BM25/keyword retrieval.
Version 10 — Reranking	Rerank retrieved candidates before generation.
Version 11 — Multi-document Comparison	Produce structured comparisons across documents.
Version 12 — Evaluation	Create a benchmark dataset and measure retrieval/generation quality.
Version 13 — Tracing	Record the full query pipeline and performance.
Version 14 — API	Expose document and query functionality through FastAPI.
Version 15 — Docker	Containerize the complete application.

8. Phase 1 — PDF Ingestion

First transform a PDF into structured records containing text and metadata.

```
{
  "document_id": "doc_001",
  "filename": "attention.pdf",
  "page": 3,
  "text": "..."
```

Preserve document ID, filename, page number, source, section when available, and later the chunk ID. Page-level metadata is essential for citations.

9. Phase 2 — Chunking

Do not embed an entire research paper as one item. Split it into retrieval units.

```
Document
↓
Chunks

Chunk 1: introduction...
Chunk 2: methodology...
Chunk 3: attention mechanism...
...
```

Start with an experimental baseline such as 500–800 tokens and 10–20% overlap. These are starting values, not universal optimums. Later compare chunk sizes and overlap using your evaluation dataset.

- Fixed-size chunking
- Recursive chunking
- Sentence-aware chunking
- Section-aware chunking
- Semantic chunking
- Parent-child retrieval

10. Phase 3 — Embeddings

Convert each chunk into a numerical vector using an embedding model. The same embedding space is used for questions and documents.

```
Question
↓
Embedding → vector Q

Chunk 1 → vector C1
Chunk 2 → vector C2
Chunk 3 → vector C3

similarity(Q, C1)
similarity(Q, C2)
similarity(Q, C3)
```

11. Phase 4 — Vector Retrieval

Start with dense retrieval before adding hybrid retrieval. A query is embedded, nearest vectors are searched, and the highest-ranked chunks are returned.

```
Question
↓
Embedding
↓
Vector search
↓
Top-K chunks
↓
Inspect scores + source pages
```

FAISS is a good learning-oriented starting point because it exposes dense-vector similarity search directly. A production-oriented vector database can be introduced later.

12. Phase 5 — Basic RAG

```
Question
↓
Retriever
```


The first retrieval stage should favor recall and speed. A reranker can spend more computation deciding which candidates are most relevant before they reach the generator.

18. Phase 11 — Multi-document Comparison

```
Question
↓
Identify relevant documents
↓
Retrieve evidence for each
↓
Rerank
↓
Structure comparison
↓
Generate answer
↓
Attach citations
```

A useful comparison output should be structured by criteria such as architecture, training objective, datasets, results, strengths, limitations, and assumptions.

19. Phase 12 — Evaluation

Evaluation turns the project from a demo into an engineering experiment. Build a curated dataset containing questions, expected evidence, and reference answers where possible.

```
[
{
  "id": "q001",
  "question": "What is self-attention?",
  "expected_sources": ["attention.pdf:p3"]
}
```

Retrieval metrics

- Recall@K
- Precision@K
- MRR — Mean Reciprocal Rank
- nDCG — normalized discounted cumulative gain

Generation metrics

- Faithfulness
- Answer relevance
- Correctness
- Citation correctness

Evaluation matrix

Configuration	Recall@5	MRR	Faithfulness	Answer relevance
Dense	Measure	Measure	Measure	Measure
BM25	Measure	Measure	Measure	Measure
Hybrid	Measure	Measure	Measure	Measure
Hybrid + reranker	Measure	Measure	Measure	Measure

Do not invent target results. The purpose of the experiments is to generate your own measurements.

20. Evaluation Dashboard

- Corpus size: documents, pages, chunks
- Retrieval: Recall@K, MRR, nDCG
- Generation: faithfulness, relevance, correctness
- Citation accuracy
- Average and P95 latency
- Embedding, retrieval, reranking, and LLM timings
- Input/output tokens and estimated cost when applicable

21. Tracing and Observability

```
QUERY
■■■■ Query preprocessing
■■■■ Embedding
■■■■ Dense retrieval
■■■■ BM25 retrieval
■■■■ Fusion
■■■■ Reranking
■■■■ Prompt construction
■■■■ LLM generation
■■■■ Citation validation
```

For each stage record useful information such as duration, candidate counts, selected chunks, model name, token usage, errors, and final latency. This makes debugging and optimization much easier.

22. API Design

Use FastAPI to expose the application.

Method	Endpoint	Purpose
GET	/health	Health check
POST	/documents	Upload and index a PDF
GET	/documents	List indexed documents
DELETE	/documents/{id}	Delete a document
POST	/query	Ask a question
POST	/compare	Compare documents
POST	/evaluate	Run evaluation
GET	/metrics	Return system/evaluation metrics

23. API Response Example

```
POST /query

{
  "question": "What is multi-head attention?"
}

Response:
```

```
{
  "answer": "...",
  "citations": [
    {
      "document": "attention.pdf",
      "page": 4,
      "chunk_id": "doc1_p4_c2"
    }
  ]
}
```

24. Docker and Deployment

Containerize the application only after the local pipeline is stable. The container should include the application, Python runtime, dependencies, and configuration. Use environment variables for secrets and deployment-specific configuration.

```
docker build .
docker run ...
```

25. Recommended Technology Stack

Layer	Initial choice	Later/alternative
Language	Python	—
PDF processing	PyMuPDF or pypdf	Specialized document parsers
Embeddings	API or Sentence Transformers	Experiment with multiple models
Vector search	FAISS	Qdrant / Pinecone / Weaviate / Milvus / pgvector
Sparse retrieval	BM25	Search engine
Reranking	Cross-encoder/reranker	Hosted reranking model
LLM	API-based LLM	Local model where appropriate
API	FastAPI	—
UI	Streamlit initially	React/Next.js
Container	Docker	Docker Compose / cloud orchestration

26. Suggested Codebase

```
advanced-rag/
├── README.md
├── LICENSE
├── pyproject.toml
├── .env.example
├── .gitignore
├── Dockerfile
├── compose.yaml
├──
├── data/
│   ├── raw/
│   ├── processed/
│   └── evaluation/
│   └──
├── src/rag/
│   ├── config/
│   ├── settings.py
│   ├── ingestion/
│   ├── loader.py
│   └── parser.py
```

```

■ ■ ■■ metadata.py
■ ■■ chunking/
■ ■ ■■ chunker.py
■ ■ ■■ strategies.py
■ ■■ embeddings/
■ ■ ■■ embedder.py
■ ■■ retrieval/
■ ■ ■■ dense.py
■ ■ ■■ sparse.py
■ ■ ■■ hybrid.py
■ ■ ■■ reranker.py
■ ■■ generation/
■ ■ ■■ prompts.py
■ ■ ■■ generator.py
■ ■■ citations/
■ ■ ■■ citation_manager.py
■ ■■ comparison/
■ ■ ■■ comparator.py
■ ■■ evaluation/
■ ■ ■■ datasets.py
■ ■ ■■ retrieval_metrics.py
■ ■ ■■ generation_metrics.py
■ ■■ tracing/
■ ■ ■■ tracer.py
■ ■■ api/
■ ■■ main.py
■ ■■ routes/
■
■ ■■ tests/
■ ■■ notebooks/

```

27. Core Data Model

```

Chunk(
  chunk_id="doc_001_p03_c02",
  document_id="doc_001",
  document_name="attention.pdf",
  page_number=3,
  section="3.2 Attention",
  text="...",
  embedding=[...]
)

```

A strong chunk object supports retrieval, metadata filtering, citations, evaluation, and tracing.

28. Prompt Architecture

Question answering

You are a research assistant.

Answer using only the supplied evidence.
 If the evidence is insufficient, say so.
 Do not invent information.
 Cite the evidence used.

Comparison

Compare the requested documents.
 For each criterion:

1. Identify evidence for each document.
2. State differences.
3. Do not infer unsupported claims.
4. Attach citations.

Keep prompts modular rather than maintaining one giant prompt for every workflow.

29. Citation Architecture

A robust approach is to label retrieved context with stable internal identifiers.

```
[DOC1-P3-C2]  
Text from the chunk...
```

```
[DOC2-P1-C4]  
Text from another chunk...
```

The backend can map these IDs to document names, pages, sections, and source URLs or file references. This is more reliable than asking an LLM to invent page numbers.

30. Failure Cases to Test

- Question is not answerable from the documents
- Retriever selects similar but incorrect evidence
- Documents contain contradictory claims
- Question asks for an exact number or equation
- Question spans multiple documents
- Question is very long
- Question is ambiguous
- Retrieved PDF text contains prompt-injection instructions
- Malformed or scanned PDFs
- Duplicate documents

31. Security Considerations

- Validate uploaded file type and size
- Limit page count and processing resources
- Treat document text as untrusted data
- Defend against prompt injection inside documents
- Protect API keys and secrets
- Use authentication and rate limits for exposed APIs
- Do not commit .env or credentials
- Log safely without exposing sensitive document content

32. Testing Strategy

Unit tests

- PDF parser
- Chunker
- Embedding interface
- Retriever
- Citation mapper
- API validation

Integration tests

- PDF → ingestion → chunking

- ## Evaluation tests

- ### 33. Experiment Plan

34. Recommended Corpus

- Transformer and LLM papers
- RAG papers
- BERT/GPT/LLaMA papers
- Robotics papers
- SLAM papers
- Drone/autonomous systems papers
- Reinforcement learning papers
- Computer vision papers

35. User Interface

Advanced RAG Research Assistant — Full Project Report

Generation

- ☐ Grounded prompting
- ☐ No-answer behavior

Citations

- ☐ Document citation
- ☐ Page citation
- ☐ Chunk citation
- ☐ Citation validation

Multi-document

- ☐ Cross-document questions
- ☐ Comparison

Evaluation

- ☐ Dataset
- ☐ Recall@K
- ☐ MRR
- ☐ nDCG
- ☐ Faithfulness
- ☐ Relevance
- ☐ Citation accuracy

Observability

- ☐ Tracing
- ☐ Latency
- ☐ Token/cost tracking

API

- ☐ /documents
- ☐ /query
- ☐ /compare
- ☐ /health

Deployment

- ☐ Dockerfile
- ☐ Environment configuration
- ☐ Container execution

38. Portfolio-Ready Outcome

Avoid describing the project as merely “a RAG chatbot using LangChain.” A stronger portfolio description emphasizes the engineering and evaluation work.

Advanced RAG Research Assistant

Designed and implemented an end-to-end research assistant for multi-document scientific PDF analysis. The system performs document ingestion, metadata-aware chunking, dense and sparse retrieval, hybrid search, reranking, grounded generation, citation attribution, and multi-document comparison. A curated evaluation dataset benchmarks retrieval and generation quality across different chunking and retrieval configurations. The system exposes a REST API through FastAPI and is containerized with Docker.

Your README should include architecture diagrams, example queries, evaluation results, experiment methodology, setup instructions, API documentation, and limitations.

39. Final Learning Sequence

1. RAG fundamentals
2. PDF processing
3. Chunking
4. Embeddings
5. Vector similarity
6. FAISS / vector database
7. Basic RAG
8. Grounded prompting
9. Citations
10. BM25
11. Hybrid retrieval
12. Reranking
13. Multi-document retrieval
14. Comparison
15. Retrieval evaluation
16. Generation evaluation
17. Tracing
18. Evaluation dashboard
19. FastAPI
20. Testing
21. Docker
22. Deployment

40. Immediate Next Step

Do not start by installing twenty libraries or implementing hybrid retrieval. Your first concrete milestone should be a command-line RAG system for one PDF.

```
paper.pdf
↓
extract text
↓
split into chunks
↓
create embeddings
↓
store vectors
↓
User: "What is the main contribution?"
↓
retrieve top 5 chunks
↓
display retrieved chunks
↓
send evidence to LLM
↓
generate answer
↓
show page citations
```

Once this works, freeze it as RAG v1.0. Then add improvements one at a time: citations → multi-document → BM25 → hybrid retrieval → reranking → comparison → evaluation → tracing → API → Docker.

41. Key Principles

- Build progressively rather than all at once.
- Do not hide the fundamentals behind frameworks at the beginning.
- Every component should solve an identifiable problem.
- Measure improvements instead of assuming they help.
- Treat retrieval as seriously as generation.
- Design provenance and citations into the data model.
- Test failure cases, not only successful questions.
- Keep a reproducible evaluation dataset.
- Only add production complexity after the core RAG pipeline works.

Final principle: The project should be treated as 10–15 progressively better versions of the same RAG system, with evaluation driving each improvement. This approach teaches RAG engineering rather than only producing a demo.

42. Reference Material

- Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” — foundational RAG paper.
- FAISS documentation — dense-vector similarity search.
- FastAPI documentation — REST API implementation.
- Docker Python guide — containerization and deployment.
- LangChain learning resources — useful later for framework-level implementations after understanding the underlying pipeline.